

Trouver l'équation d'une décision

Fiche de formation · régression symbolique, programmation génétique, Architecte et Oracle · 17/09/2026

1 Le problème

Dans chaque vignette, le détachement choisit entre deux options : partir ou se terrorer, direct ou détour.

On cherche la **règle** qui dit dans quelle situation l'option 1 vaut mieux.

On la veut sous forme d'**équation lisible**, pour pouvoir la vérifier, la comprendre et la contredire. Un réseau de neurones pourrait bien décider, mais sans dire pourquoi.

2 Régression classique ou symbolique

- **Classique** : on fixe la forme, par exemple une droite $y = a \cdot x + b$, et on cherche seulement les nombres a et b .
- **Symbolique** : on cherche la **forme et les nombres**. La réponse peut être $3 \cdot (0,5 - \text{distance})$, $\min(\text{distance} ; 0,4) \times \text{alarme}$ ou « si alarme et distance < 0,7 ».
- **Le prix** : l'espace des formules est immense. Il faut une méthode de recherche, et une règle qui punit les formules trop longues.

3 L'astuce : la pièce de monnaie

L'option est tirée à pile ou face ($p = 1/2$). Pour chaque épisode, on fabrique une **pseudo-issue** :

$$\psi = 2 \cdot Y \cdot (2a - 1)$$

En clair : ψ vaut **+2** si l'option 1 a réussi, **-2** si l'option 0 a réussi, **0** en cas d'échec.

En moyenne, dans une situation x :

$$E[\psi \mid x] = P(\text{réussite} \mid \text{option 1}) - P(\text{réussite} \mid \text{option 0}) = \tau(x)$$

- N'importe quel algorithme de régression appliqué à ψ cherche donc directement l'**avantage $\tau(x)$** .
- La règle devient : **option 1 si la formule est positive**.
- La pièce décide seule, donc l'effet est **causal** : il ne vient pas de qui a choisi quoi.

4 Une formule doit payer sa place

$$f^* = \operatorname{argmin} [\text{erreur}(f) + \lambda \cdot |f|]$$

$|f|$ = nombre de symboles de la formule

- Chaque symbole coûte λ . Un terme n'est gardé que s'il réduit l'erreur **plus qu'il ne coûte**.
- C'est le rasoir d'Occam écrit en mathématiques (longueur de description minimale).

- Sans cette pénalité, l'algorithme **apprend le bruit** : une formule compliquée qui colle aux épisodes d'hier et se trompe demain.

5 Juger un algorithme : il doit savoir échouer

- **Données jamais vues** : la règle est apprise sur 4 parts des épisodes et jugée sur la 5^e, ou sur un monde laissé de côté.
- **Gain sur la meilleure option fixe**, qui doit être nettement positif :

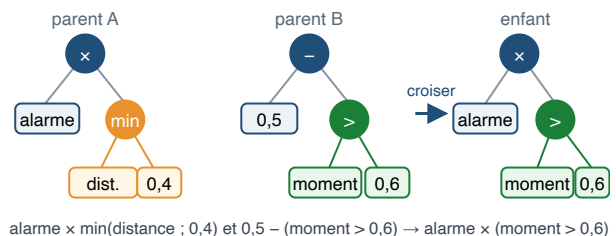
$$G = V(\text{règle}) - \max(V(\text{option 0}), V(\text{option 1}))$$

- **Contrôle positif** : une formule cachée, connue de nous, doit être retrouvée.
- **Contrôle nul** : sans formule cachée, l'algorithme ne doit rien inventer (au plus 1 fois sur 20).
- **Leurres** : des variables sans effet, dont une corrélée à une vraie, ne doivent pas entrer dans la formule.

Leçon du 17/09. Calibrer **avant** de juger. Il faut vérifier que la **bonne** règle serait détectée avec ce nombre d'épisodes. Avec les premiers réglages du banc, la vraie formule en croix n'était détectée que **9 fois sur 100** à 500 épisodes : le banc n'aurait rien départagé.

6 La programmation génétique

C'est la sélection naturelle appliquée aux équations (Koza, 1992). Une formule est un **arbre** : les nœuds sont des opérations, les feuilles des variables ou des constantes.



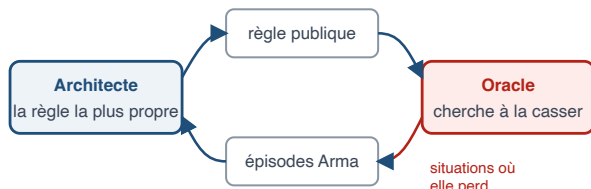
- **Naissance** : 500 à 5000 formules tirées au hasard.
- **Sélection** : on garde les plus justes, pénalisées par leur longueur.
- **Croisement** : deux parents échangent un sous-arbre.
- **Mutation** : un symbole change au hasard.
- On recommence pendant 15 à 100 **générations**.

EvoGP fait exactement cela, mais range toute la population dans des tableaux de taille fixe et l'évalue **d'un seul coup sur la carte graphique**. Ses auteurs annoncent jusqu'à 100 fois plus vite.

7 Les familles d'algorithmes

famille	idée	exemples	machine
Linéaire parcimonieux	une frontière droite ; la pénalité L1 met à zéro les coefficients inutiles	logistique L1	CPU, 1 s
Arbre	une suite de questions : « distance < 0,4 ? »	arbre de régression	CPU, 1 s
Programmation génétique	une population de formules évolue (§ 6)	gplearn, PySR, EvoGP	CPU ; EvoGP sur GPU
Réseau symbolisé	un réseau est appris, puis chaque lien est remplacé par une fonction simple	KAN, EQL	GPU
Transformeur pré-entraîné	lit le nuage de points et écrit la formule d'un coup ; fragile au bruit	E2E, TPSR	GPU
Modèle de langage	propose des squelettes de formules, puis on ajuste les constantes	LLM-SR, FunSearch	GPU
Renforcement	un agent écrit la formule symbole par symbole ; récompense = ajustement	DSR, Φ -SO	GPU ou CPU

8 L'Architecte et l'Oracle



- L'**Architecte** apprend la règle la plus simple sur tous les épisodes.
- L'**Oracle** cherche à la **casser au maximum**. Il envoie les épisodes suivants là où le **regret** est grand :

$$p(\theta) = \max(V_0(\theta), V_1(\theta)) - V_{\text{règle}}(\theta)$$

En clair : casser, c'est faire perdre la règle **là où une autre option aurait réussi**. Un monde impossible (tout échoue) rapporte $p = 0$: l'Oracle n'y gagne rien, et l'Architecte n'y apprendrait rien.

L'Oracle **voit ce que le détachement ne voit pas** : la situation vraie θ . La valeur de cette information cachée est :

$$VI(x) = E[\max_a \mu_a(\theta) \mid x] - \max_a E[\mu_a(\theta) \mid x] \geq 0$$

En clair : c'est ce que gagnerait un détachement qui verrait θ . Elle n'est jamais négative, car la moyenne des maximums dépasse toujours le maximum des moyennes.

Poussé par l'Oracle, l'**Architecte se dépasse en trois niveaux** :

niveau	quand	ce qui change
1. coefficients	la faille se corrige en réajustant	les nombres de la formule
2. forme	la faille résiste, et $VI \approx 0$	un terme nouveau , s'il paie sa place (§ 4)
3. perception	VI grande : la faille est invisible	une perception ou une option nouvelle

Le progrès se mesure par la **pire faille confirmée** à chaque tour, qui doit baisser. Une **archive** des anciennes failles empêche l'Architecte de les rouvrir.

9 Pièges à retenir

- Le maximum de moyennes bruitées est trop optimiste.** Une faille se confirme sur des épisodes neufs.
- Une variable corrélée** à la vraie peut prendre sa place dans la formule.
- « **Indifférent** » veut dire « **plus petit que ce qu'on sait voir** », pas « nul ».
- Une équation ne décide pas selon ce qui n'est pas perçu.** Au 17/09, au moment du choix, le détachement ne voyait presque rien de la menace. En P2, seul « véhicule vu » variait (8 épisodes sur 128) ; en P4, rien.

Où en est le banc (17/09). Test B, sur les vrais épisodes Arma : **aucune règle** ne bat la meilleure option fixe, comme attendu. Une piste non établie sur le pilote P5 : « alarme depuis plus de 7,5 s → repartir à 45 s », +6 points, IC [-1 ; +13,5]. Test A, avec des formules cachées : L1 et arbre finis, gplearn en cours, **EvoGP** (GPU) en installation.

Pour aller plus loin. Koza (1992), *Genetic Programming*. Schmidt et Lipson (2009), *Science*, lois physiques retrouvées par évolution. Cranmer (2023), PySR. Petersen et al. (2021), Deep Symbolic Regression. Kamienny et al. (2022), régression symbolique de bout en bout par transformeurs. Liu et al. (2024), KAN. Shojaaee et al. (2024), LLM-SR. EMI-Group (2025), EvoGP, arXiv 2501.17168. Dennis et al. (2020), PAIRED : générer des environnements par le regret. Dépôt : [plans/plan-architecte-oracle.md](#), [equation/CRITERES_BANC_EQUATION.md](#).